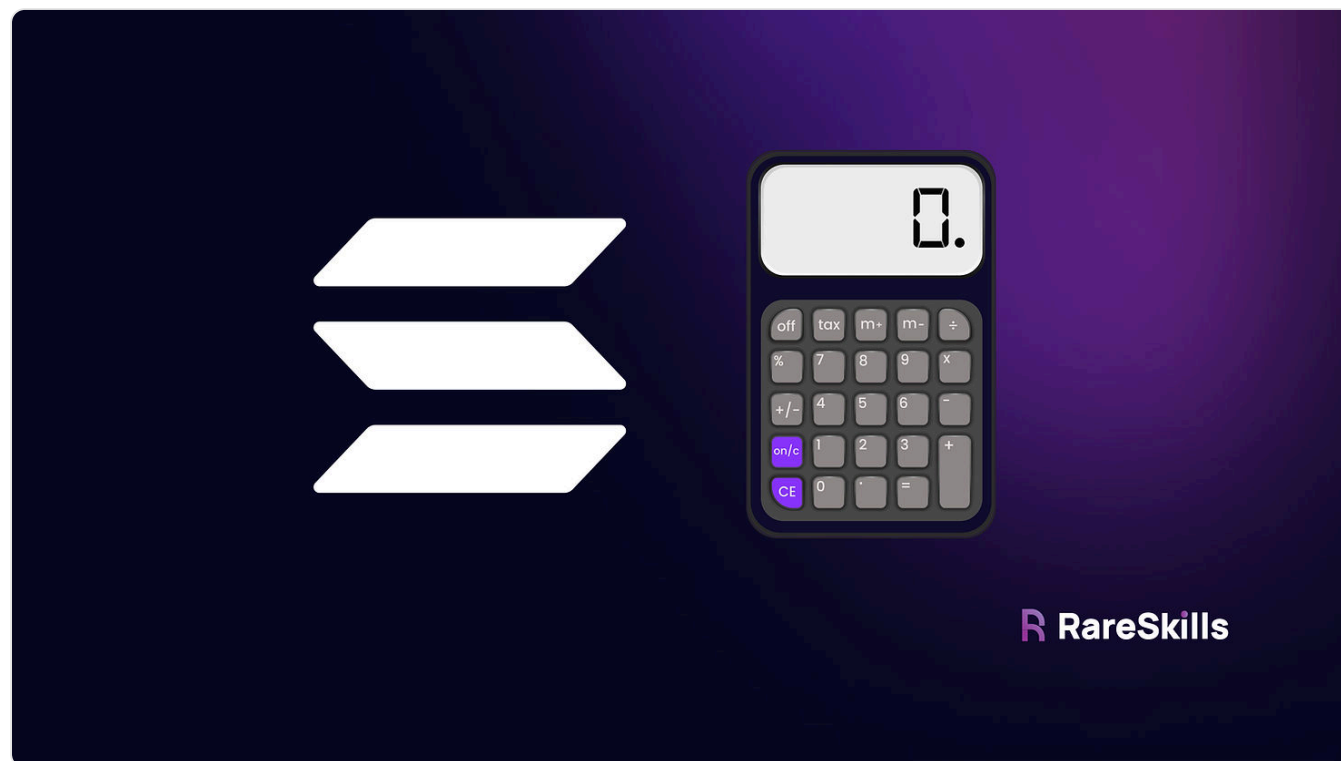




Blog / [Arithmetic and Basic Types in Solana and Rust](#)

Arithmetic and Basic Types in Solana and Rust



```
contract Day2 {  
  
    event Result(uint256);  
    event Who(string, address);  
  
    function doSomeMath(uint256 a, uint256 b) public {  
        uint256 result = a + b;  
        emit Result(result);  
    }  
  
    function sayHelloToMe() public {  
        emit Who("Hello World", msg.sender);  
    }  
}
```

[Copy](#)

Let's start a new project

```
anchor init day2  
cd day2  
anchor build  
anchor keys sync
```

[Copy](#)

Be sure you have the Solana test validator running in one terminal:

And the Solana logs in another:

```
solana logs
```

Copy

Make sure the newly scaffolded program works properly by running the tests

```
anchor test --skip-local-validator
```

Copy



On This Page

Supplying Function Arguments

Math in Solana

Arithmetic Overflow

How does Solana defend against arithmetic overflow?

Solana compute units 101

Powers does not use the same syntax as Solidity

Floating points

Supplying Function Arguments

Before we do any math, let's change the initialize function to receive two integers. Ethereum uses uint256 as the "standard" integer size. On Solana, it is u64 — this is equivalent to uint64 in Solidity.

Passing unsigned integers

The default initialize function will look like the following:

```
pub fn initialize(ctx: Context<Initialize>) → Result<()> {  
    Ok(())  
}
```

Copy

Modify `initialize()` function in `lib.rs` to be as follows.

[Courses](#) ▾[Blog](#)[Testimonials](#)[Web3 Jobs](#)[Pricing](#)[Apply Now](#)

```
pub fn initialize(ctx: Context<Initialize>,
                a: u64,
                b: u64) → Result<()> {
    msg!("You sent {} and {}", a, b);
    Ok(())
}
```

Now we need to change the test in `./tests/day2.ts`.

```
it("Is initialized!", async () => {
    // Add your test here.
    const tx = await program.methods
        .initialize(new anchor.BN(777), new anchor.BN(888)).rpc();
    console.log("Your transaction signature", tx);
});
```

[Copy](#)

Now re-run `anchor test --skip-local-validator`.

When we look in the logs, we should see something like the following

```
Transaction executed in slot 367357:
  Signature:
  54iJFbtEE61T9X2WCLbMe8Dq2YYBzCLYE4qW2DqTsA4gZRgootcubLgHc1MHYncbP63sNxEX8t
  Status: Ok
  Log Messages:
    Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY invoke [1]
```

[Copy](#)

Program log: Instruction: Initialize

Program log: You sent 777 and 888

Courses ▾

Blog

Testimonials

Web3 Jobs

Pricing



Apply Now

Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY consumed

200000 compute units

Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY success

Passing a string

Now let's illustrate how to pass a string as an argument.

```
pub fn initialize(ctx: Context<Initialize>,
    a: u64,
    b: u64,
    message: String) → Result<()> {
    msg!("You said {:?}", message);
    msg!("You sent {} and {}", a, b);
    Ok(())
}
```

Copy

And change the test.

```
it("Is initialized!", async () => {
    // Add your test here.
    const tx = await program.methods
        .initialize(
            new anchor.BN(777), new anchor.BN(888), "hello").rpc();
```

Copy

When we run the test, we see the new log.

Array of numbers

Next we add a function (and test) to illustrate passing an array of numbers. In Rust, a "vector", or `Vec` is what Solidity calls an "array."

```
pub fn initialize(ctx: Context<Initialize>,  
    a: u64,  
    b: u64,  
    message: String) → Result<()> {  
    msg!("You said {:?}", message);  
    msg!("You sent {} and {}", a, b);  
    Ok(())  
}  
  
// added this function  
pub fn array(ctx: Context<Initialize>,  
    arr: Vec<u64>) → Result<()> {  
    msg!("Your array {:?}", arr);  
    Ok(())  
}
```

[Copy](#)

And we update the unit test as follows

```
it("Is initialized!", async () => {  
  // ...  
  const tx = await program.methods.initialize(new anchor.BN(777), new  
anchor.BN(888), "hello").rpc();  
  console.log("Your transaction signature", tx);  
});  
  
// added this test  
it("Array test", async () => {  
  const tx = await program.methods.array([new anchor.BN(777),  
new anchor.BN(888)]).rpc();  
  console.log("Your transaction signature", tx);  
});
```

[Copy](#)[Courses](#)[Blog](#)[Testimonials](#)[Web3 Jobs](#)[Pricing](#)[Apply Now](#)

And we run the tests again and view the logs to see the array output:

```
Transaction executed in slot 368489:  
  Signature:  
3TBzE3NddEY8KREv1FSXnieoyT6G6iNxF1n4hJHCeeWhAsUward3MEKm9WJHV4PMjPxeN2jRSR0  
  Status: Ok  
  Log Messages:  
    Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY invoke [1]  
    Program log: Instruction: Initialize  
    Program log: You said [777, 888]  
    Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY consumed 1587 of  
200000 compute units  
    Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY success
```

[Copy](#)



Math in Solana

Floating point math

Solana has some, although limited, native support for floating point operations.

However, it's best to avoid floating point operations because of how computationally intensive they are (we will see an example of this later). Note that Solidity has *no* native support for floating point operations.

Read more about the limitations of using floats [here](#).

Arithmetic Overflow

Arithmetic overflow was a common attack vector in Solidity until version 0.8.0 built overflow protection into the language by default. In Solidity 0.8.0 or higher, the overflow checks are done by default. Since these checks consume gas, sometimes devs strategically disable them with the "unchecked" block.

Method 1: `overflow-checks = true` in `Cargo.toml`

If the key `overflow-checks` is set to `true` in the `Cargo.toml` file, then Rust will add overflow checks at the compiler level. See the screenshot of `Cargo.toml` next:

```
[workspace]
members = [
  "programs/*"
]

[profile.release]
overflow-checks = true
lto = "fat"
codegen-units = 1
[profile.release.build-override]
opt-level = 3
incremental = false
codegen-units = 1
```

If the `Cargo.toml` file is configured in this manner, you don't need to worry about overflow.

However, adding overflow checks increases the compute cost of the transaction (we will revisit this shortly). So under some circumstances where compute cost is an

Method 2: using `checked_*` operators.

Let's look at how overflow checks are applied to arithmetic operations within Rust itself. Consider the snippet of Rust below.

- On line 1, we do arithmetic using the usual `+` operator, which overflows silently.
- On line 2, we use `.checked_add`, which will throw an error if an overflow happens. Note that we have `.checked_*` available for other operations, like `checked_sub` and `checked_mul`.

```
let x: u64 = y + z; // will silently overflow
let xSafe: u64 = y.checked_add(z).unwrap(); // will panic if
overflow happens

// checked_sub, checked_mul, etc are also available
```

[Copy](#)

Exercise 1: Set `overflow-checks = true` create a test case where you underflow a `u64` by doing `0 - 1`. You will need to pass those numbers in as arguments or the code won't compile. What happens?

You'll see the transaction fails (with a rather cryptic error message shown below) when the test runs. That's because Anchor turned on overflow protection:

```

day_2
1) Is initialized!
0 passing (63ms)
1 failing

1) day_2
  Is initialized!:
  Error: failed to send transaction: Transaction simulation failed: Error processing Instruction 0: Program failed to complete
    at Connection.sendEncodedTransaction (node_modules/@solana/web3.js/src/connection.ts:5920:13)
    at processTicksAndRejections (node:internal/process/task_queues:95:5)
    at Connection.sendRawTransaction (node_modules/@solana/web3.js/src/connection.ts:5879:20)
    at sendAndConfirmRawTransaction (node_modules/@project-serum/anchor/src/provider.ts:288:21)
    at AnchorProvider.sendAndConfirm (node_modules/@project-serum/anchor/src/provider.ts:148:14)
    at MethodsBuilder.rpc [as _rpcFn] (node_modules/@project-serum/anchor/src/program/namespace/rpc.ts:29:16)

error Command failed with exit code 1.

```

Exercise 2: Now change `overflow-checks` to `false`, then run the test again. You should see an underflow value of 18446744073709551615.

Exercise 3: With overflow protection disabled in Cargo.toml, do `let result = a.checked_sub(b).unwrap();` with `a = 0` and `b = 1`. What happens?

Should you just leave `overflow-checks = true` in the Cargo.toml file for your Anchor project? Generally, yes. But if you are doing some intensive calculations, you might want to set `overflow-checks` to `false` and strategically defend against overflows in key junctures to save compute cost, which we will demonstrate next.

Solana compute units 101

In Ethereum, a transaction runs until it consumes the “gas limit” specified by the transaction. Solana calls “gas” a “compute unit.” By default, a transaction is limited to

Determining compute costs of a transaction in Solana

Solana is indeed cheap to use compared to Ethereum, but that does not mean your optimization skills in Ethereum development are useless. Let's measure how much compute units our math functions require.

The Solana logs terminal also shows how many compute units were used. We've provided benchmarks for the checked and unchecked subtraction below.

With overflow protection disabled consumes 824 compute units:

```
Transaction executed in slot 371694:
  Signature: iN5YGdMH1Z6NDcCbNkrJnETtKwPntBbWXgxiwGTrv3yBXvVg58HyWqMgD18DFSGWJTzXg85RDZcUszdgWf
  LRPnx
  Status: Ok
  Log Messages:
    Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY invoke [1]
    Program log: Instruction: Initialize
    Program log: Result = 8
    Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY consumed 824 of 200000 compute units
    Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY success
```

With overflow protection enabled in consumes 872 compute units:

```
Transaction executed in slot 371868:
Signature: 4LjNWqCpmD2HNycxLc16byWQZ2PSH4VXmoLTwd9cr7dRTtBqLyyhiTHXDFi5ugMuXPZjd5B6FK8b6aFIHz
UddLeS
Status: OK
Log Messages:
Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY invoke [1]
Program log: Instruction: Initialize
Program log: Result = 8
Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY consumed 872 of 200000 compute units
Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY success
```

As you can see, just doing a simple math operation takes up almost 1000 units. Since we have 200k units, we can only do a few hundred simple arithmetic operations within the per-transaction gas limit. So, while transactions on Solana are generally cheaper than on Ethereum, we are still limited by a relatively small compute unit cap and would not be able to carry out computationally intensive tasks like fluid dynamic simulations on the Solana chain.

We'll revisit transaction cost later.

Powers does not use the same syntax as Solidity

In Solidity, if we want to raise `x` to the `y` power, we do

```
uint256 result = x ** y;
```

[Copy](#)

Rust does not use this syntax. Instead, it uses `.pow`

There is also `.checked_pow` if you are concerned about overflow.

Floating points

One nice thing about using Rust for smart contracts is that we don't have to import libraries like Solmate or Solady to do math. Rust is a pretty sophisticated language with a lot of operations built in, and if we need some piece of code, we can look outside the Solana ecosystem for a Rust crate (this is what libraries are called in Rust) to do the job.

Let's take the cube root of 50. The cube root function for floats is built into the Rust language with the function `cbrt()`.

```
// note that we changed `a` to f32 (float 32)
// because `cbrt()` is not available for u64
pub fn initialize(ctx: Context<Initialize>, a: f32) → Result<()> {
    msg!("You said {:?}", a.cbrt());
    Ok(());
}
```

Copy

Remember how we said in an earlier section that floats can be computationally expensive? Well, here we see our cube root operation consumed over 5 times as

Transaction executed in slot unspecified:

Signature:

VfvvSG5vvVSAAnsYLCsvB9N6PsuGwL39kKd1fMsyvuB7y5DUHURwQVHU9rv3Xkz5NJqGHLSXoWoV

Status: Ok

Log Messages:

Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY invoke [1]

Program log: Instruction: Initialize

Program log: attempting to begin the function with 50

Program log: Result = 3.6840315

Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY consumed 4860 of 200000 compute units

Program 8o3ehd3XnyDocd9hG1uz5trbmSRB7gaLaE9BCXDpEnMY success

Exercise 4: Build a calculator that does the +, -, x, and ÷. and also sqrt and log10.

Originally Published February, 09, 2024

Looking For
An Audit?

RareSkills

Leverage our extensive

network of top security
specialists.

Get A Quote 



YieldNest

Courses 

Blog

Testimonials

Web3 Jobs

Pricing

Senior Smart
Contract Engineer

Join YieldNest as a
Senior Smart Contract
Engineer to architect and
deploy...

Apply Now >

Foreverlong

Join us as a Full Stack
Developer to architect
and ship a...

Apply Now >



MONAD



Senior
Engineer

Apply Now

looking for a Senior
Frontend Engineer who
lives and breathes
interface quality...

Apply Now >

Join Our Newsletter

Occasional updates straight to your inbox. No spam.

Your email address

Subscribe

RareSkills

The Web3 education platform trusted by
industry-leading developers and auditors.



Blog

Learn Solidity

Events

Testimonials

60 Days of Solana

Contact

Web3 Jobs

ZK Book

